

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve, classification_report, precision_recall_curve
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold, GridSearchCV, RandomizedSearchCV

from collections import Counter
```

C:\Users\Prashant\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).

```
from pandas.core import (
```

```
In [2]: df = pd.read_csv("churn_prediction.csv")
df.shape
```

Out[2]: (28382, 21)

```
In [3]: df.head()
```

Out[3]:

	customer_id	vintage	age	gender	dependents	occupation	city	customer_nw_category	branch_code	current_balance	...	average
0	1	2101	66	Male	0.0	self_employed	187.0	2	755	1458.71	...	
1	2	2348	35	Male	0.0	self_employed	NaN	2	3214	5390.37	...	
2	4	2194	31	Male	0.0	salaried	146.0	2	41	3913.16	...	
3	5	2329	90	NaN	NaN	self_employed	1020.0	2	582	2291.91	...	
4	6	1579	42	Male	2.0	self_employed	1494.0	3	388	927.72	...	

5 rows × 21 columns

```
In [4]: df['churn'].value_counts()
```

```
Out[4]: churn
0      23122
1       5260
Name: churn, dtype: int64
```

Data Preprocessing

```
In [5]: df.isnull().sum()
```

```
Out[5]: customer_id      0
vintage      0
age      0
gender      525
dependents    2463
occupation     80
city      803
customer_nw_category  0
branch_code     0
current_balance  0
previous_month_end_balance  0
average_monthly_balance_prevQ  0
average_monthly_balance_prevQ2  0
current_month_credit  0
previous_month_credit  0
current_month_debit  0
previous_month_debit  0
current_month_balance  0
previous_month_balance  0
churn      0
last_transaction  0
dtype: int64
```

```
In [ ]:
```

```
In [ ]:
```

```
In [6]: df['occupation'].mode()
```

```
Out[6]: 0    self_employed
Name: occupation, dtype: object
```

```
In [7]: df["city"].mode()
```

Out[7]: 0 1020.0
Name: city, dtype: float64

a) Missing Value Treatment

```
In [50]: #Convert Gender
df["gender"] = df["gender"].map({"Male":0, "Female":1})

# Replace with -1 for missing gender
df['gender'] = df['gender'].fillna(-1)

# Replacing with max. occurrence values
df['dependents'] = df['dependents'].fillna(0)
df['occupation'] = df['occupation'].fillna('self_employed')
df['city'] = df['city'].fillna(1020)
```

In []:

In [51]: df.head()

Out[51]:

	customer_id	vintage	age	gender	dependents	occupation	city	customer_nw_category	branch_code	current_balance	...	average
0	1	2101	66	0.0	0.0	self_employed	187.0	2	755	1458.71	...	
1	2	2348	35	0.0	0.0	self_employed	1020.0	2	3214	5390.37	...	
2	4	2194	31	0.0	0.0	salaried	146.0	2	41	3913.16	...	
3	5	2329	90	-1.0	0.0	self_employed	1020.0	2	582	2291.91	...	
4	6	1579	42	0.0	2.0	self_employed	1494.0	3	388	927.72	...	

5 rows × 21 columns

```
In [52]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
categorical=df.select_dtypes(include="object")
for i in categorical:
    df[i]=le.fit_transform(df[i])
```

Train and Test Split

```
In [53]: #x = df.drop('Attrition', axis=1)
from sklearn.model_selection import train_test_split
X = df.drop(['churn', 'customer_id'], axis=1)
y = df['churn']
# Splitting the data into train and test
```

```
In [54]: from sklearn.preprocessing import StandardScaler
Scaler_X = StandardScaler()
X_sc = Scaler_X.fit_transform(X)
```

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(X_sc, y, test_size=0.2, random_state=100)
```

```
In [ ]: X_train
```

Handling class imbalance using SMOTE based techniques

A) SMOTE Technique

```
In [55]: from imblearn.over_sampling import SMOTE
from collections import Counter

counter = Counter(y_train)
print('Before', counter)
# oversampling the train dataset using SMOTE
smt = SMOTE()
#X_train, y_train = smt.fit_resample(X_train, y_train)
X_train_sm, y_train_sm = smt.fit_resample(X_train, y_train)

counter = Counter(y_train_sm)
print('After', counter)
```

Before Counter({0: 18495, 1: 4210})

After Counter({0: 18495, 1: 18495})

```
In [ ]: X_train_sm
```

B) ADASYN Technique

```
In [56]: from imblearn.over_sampling import ADASYN

counter = Counter(y_train)
print('Before', counter)
# oversampling the train dataset using ADASYN
ada = ADASYN(random_state=130)
X_train_ada, y_train_ada = ada.fit_resample(X_train, y_train)

counter = Counter(y_train_ada)
print('After', counter)
```

Before Counter({0: 18495, 1: 4210})

After Counter({0: 18495, 1: 17978})

Model Building - Imbalanced data

```
In [58]: from sklearn.metrics import classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
log_model=LogisticRegression()
log_model.fit(X_train,y_train)
y_pred=log_model.predict(X_test)
print(classification_report(y_pred,y_test))
print()
print(confusion_matrix(y_pred,y_test))
```

	precision	recall	f1-score	support
0	0.99	0.82	0.90	5576
1	0.07	0.70	0.12	101
accuracy			0.82	5677
macro avg	0.53	0.76	0.51	5677
weighted avg	0.98	0.82	0.89	5677

```
[[4597  979]
 [   30   71]]
```

C:\Users\Prashant\AppData\Roaming\Python\Python310\site-packages\sklearn\linear_model_logistic.py:444: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
In [59]: from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
log_model=LogisticRegression()
log_model.fit(X_train_sm,y_train_sm)
y_pred=log_model.predict(X_test)
print(classification_report(y_pred,y_test))
```

	precision	recall	f1-score	support
0	0.79	0.91	0.84	4011
1	0.66	0.41	0.51	1666
accuracy			0.76	5677
macro avg	0.72	0.66	0.68	5677
weighted avg	0.75	0.76	0.75	5677

C:\Users\Prashant\AppData\Roaming\Python\Python310\site-packages\sklearn\linear_model_logistic.py:444: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

```
In [60]: from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
log_model=LogisticRegression()
log_model.fit(X_train_ada,y_train_ada)
y_pred=log_model.predict(X_test)
print(classification_report(y_pred,y_test))
```

	precision	recall	f1-score	support
0	0.81	0.91	0.86	4103
1	0.65	0.44	0.52	1574
accuracy			0.78	5677
macro avg	0.73	0.67	0.69	5677
weighted avg	0.76	0.78	0.76	5677

C:\Users\Prashant\AppData\Roaming\Python\Python310\site-packages\sklearn\linear_model_logistic.py:444: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

Model-1: Logistic Regression

Model-2: Decision Tree

```
In [61]: estimators = [2,10,30,50,100]
# Maximum number of depth in each tree:
max_depth = [i for i in range(5,16,2)]
# Minimum number of samples to consider to split a node:
min_samples_split = [2, 5, 10, 15, 20, 50, 100]
# Minimum number of samples to consider at each leaf node:
min_samples_leaf = [1, 2, 5]
```

1. Original Unsampled Data

```
In [62]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold, GridSearchCV, RandomizedSearchCV

tree_model = DecisionTreeClassifier()

tree_param_grid = {
    'max_depth': max_depth,
    'min_samples_split': min_samples_split,
    'min_samples_leaf': min_samples_leaf
}

clf_DT = RandomizedSearchCV(tree_model, tree_param_grid, cv=5, n_jobs=-1)
clf_DT.fit(X_train, y_train)
clf_DT.best_estimator_
```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

```
Out[62]: ▼ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=7, min_samples_split=100)
```

```
In [63]: dtc=DecisionTreeClassifier(max_depth=11,min_samples_split=100)
dtc.fit(X_train,y_train)
y_pred=dtc.predict(X_test)
print(classification_report(y_pred,y_test))
```


	precision	recall	f1-score	support
0	0.94	0.88	0.91	4912
1	0.45	0.62	0.53	765
accuracy			0.85	5677
macro avg	0.70	0.75	0.72	5677
weighted avg	0.87	0.85	0.86	5677

2.SMOTE Resampling

```
In [64]: clf_DT.fit(X_train_sm, y_train_sm)
         clf_DT.best_estimator_
```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

```
Out[64]: ▾ DecisionTreeClassifier
         DecisionTreeClassifier(max_depth=9, min_samples_split=100)
```

```
In [65]: dtc=DecisionTreeClassifier(max_depth=9,min_samples_split=50)
         dtc.fit(X_train_sm,y_train_sm)
         y_pred=dtc.predict(X_test)
         print(classification_report(y_pred,y_test))
```

	precision	recall	f1-score	support
0	0.80	0.91	0.85	4064
1	0.65	0.42	0.51	1613
accuracy			0.77	5677
macro avg	0.72	0.67	0.68	5677
weighted avg	0.76	0.77	0.75	5677